โครงงานวิชา AIE313 Database System (Mini Project)

คะแนนรวม 20 **คะแนน** (+ 5 **คะแนน**)

ระยะเวลาที่กำหนด

- กำหนดวันนำเสนอผลงาน: วันที่ 11 พฤศจิกายน - 9 ธันวาคม 2569
- กำหนดส่งรายงานฉบับสมบูรณ์ (วันสุดท้าย): วันพุธที่ 9 ธันวาคม 2569

ขอบเขตของโครงงาน

- งานกลุ่ม (ไม่เกิน 3 คน) พัฒนา ระบบจัดการข้อมูล (Information Management System) ที่ใช้ฐานข้อมูล Supabase เป็นหลัก
- สร้างระบบ Web Application โดยใช้ภาษา HTML, JavaScript และ Supabase หรือเครื่องมืออื่นๆ ที่เกี่ยวข้อง Deploy ลง Web Hosting จริง เช่น Firebase Hosting หรือ Vercel เป็นต้น
- ต้องมี การแปลงความหมายระดับภายนอก/ระดับแนวคิด, ER-Diagram, Relational Schema, Views, การ Normalization (จำลองข้อมูลได้เอง), SQL Script, และส่วนติดต่อผู้ใช้ (UI)
- ต้องมี ส่วน Chat ที่เชื่อมต่อ LLM เข้ากับฐานข้อมูล Supabase เพื่อให้ผู้ใช้สอบถามข้อมูลด้วยภาษาธรรมชาติ (ภาษาไทย/อังกฤษ)

แนวทางของโครงงาน

ให้นักศึกษาทำ Project ที่เกี่ยวข้องกับฐานข้อมูล เพื่อการสร้างระบบ Web Application โดยมีข้อกำหนดดังนี้

ข้อกำหนดฐานข้อมูล

- กำหนดให้สมาชิกแต่ละคนในกลุ่มมีการออกแบบระบบฐานข้อมูลเป็นของตัวเอง เช่น การแปลงความหมายระดับภายนอก/ระดับแนวคิด, ER-Diagram, Relational Schema, Views, SQL Script
- กำหนดให้นำระบบฐานข้อมูลของสมาชิกแต่ละคนมารวมกันเป็นฐานข้อมูลเดียวกัน โดยต้องมีตารางความสัมพันธ์กัน เช่น ตาราง user เดียวกัน
- ระบบฐานข้อมูลต้องมีตารางข้อมูลไม่น้อยกว่า 10 ตาราง และ View ไม่น้อยกว่า 6 Views
- ข้อมูลจำลองในแต่ละตารางหลักไม่น้อยกว่า 30 แถว เพื่อให้การสอบถามผ่าน Chat และการทำ Aggregation มีความหมาย

ข้อกำหนดระบบ Web Application

- ระบบต้องมีหน้า Registration/Login เพื่อเข้าสู่หน้าเว็บได้ (เช่น Supabase Auth)
- ระบบสามารถ เพิ่ม/แก้ไข/ลบ ข้อมูลในตาราง (CRUD)
- ระบบต้องสามารถแสดงผลข้อมูลจาก View ได้
- ระบบต้องมีหน้า Chat สำหรับสอบถามข้อมูลจากฐานข้อมูล (รายละเอียดตามหัวข้อถัดไป)

ข้อกำหนดการใช้งานภาษา SQL สำหรับการวิเคราะห์ข้อมูล

ให้เขียนคำสั่ง SQL บนฐานข้อมูลของกลุ่ม (PostgreSQL บน Supabase) ครบทั้ง 4 กลุ่มด้านล่าง กลุ่มละไม่น้อยกว่า 4 คำสั่ง รวมไม่น้อยกว่า 16 คำสั่ง ทุกคำสั่งต้องตอบคำถามทางธุรกิจที่มีความหมาย และ View ที่กำหนด (≥6 Views) ควรสร้างจากคำสั่งในกลุ่มเหล่านี้

SQL: Select Join Tables

- INNER JOIN อย่างน้อย 1 คำสั่ง ที่เชื่อม 3 ตารางขึ้นไป

SQL: Aggregate Functions

- ใช้ COUNT(*), COUNT(DISTINCT ...), SUM, AVG, MIN, MAX ให้ครบทุกฟังก์ชัน
- ใช้ Aggregate ร่วมกับ JOIN อย่างน้อย 1 คำสั่ง
- ใช้ Aggregate ร่วมกับ CASE WHEN อย่างน้อย 1 คำสั่ง เช่น นับจำนวนคำสั่งซื้อแยกตามสถานะในแถวเดียว

  ตัวอย่าง: SELECT COUNT(DISTINCT customer_id), SUM(total), AVG(total) FROM orders;

SQL: Grouping Functions

- GROUP BY อย่างน้อย 1 คำสั่ง ที่จัดกลุ่มมากกว่า 1 คอลัมน์
- HAVING เพื่อกรองผลหลังจัดกลุ่ม อย่างน้อย 1 คำสั่ง
- Function เช่น SUM() MAX,MIN (... ORDER BY ...) อย่างน้อย 1 คำสั่ง

  ตัวอย่าง: SELECT category, region, SUM(total) FROM v_sales GROUP BY ROLLUP (category, region);

Statistical and Mathematical Functions

- ฟังก์ชันทางสถิติ: STDDEV, VARIANCE, MODE() หรืออื่นๆ อย่างน้อย 2 ฟังก์ชัน

การนำเสนอผลการวิเคราะห์ในรายงาน (ทุกคำสั่ง)

- คำถามทางธุรกิจ | คำสั่ง SQL | ภาพผลลัพธ์ | การแปลความหมายของผลลัพธ์
- ผลการวิเคราะห์ที่สำคัญอย่างน้อย 3 รายการ ต้องแสดงบนหน้าเว็บ (เช่น Dashboard จาก View) และสามารถถามผ่านระบบ Chat ได้

ข้อกำหนดส่วน Chat เชื่อมต่อ LLM กับ Supabase (5 คะแนน)

วัตถุประสงค์

ให้นักศึกษาสร้างหน้า Chat บน Web Application ที่ผู้ใช้พิมพ์คำถามเป็นภาษาธรรมชาติ แล้วระบบใช้ LLM แปลงคำถามเป็นการดึงข้อมูลจากฐานข้อมูล Supabase ของกลุ่ม และตอบกลับเป็นข้อความพร้อมตารางผลลัพธ์

ความต้องการขั้นต่ำ

1. มีหน้า Chat UI แสดงประวัติการสนทนา (คำถาม/คำตอบ), ช่องพิมพ์, สถานะกำลังประมวลผล
2. คำตอบต้องมาจากข้อมูลจริงในฐานข้อมูลของกลุ่ม ไม่ใช่ LLM เดาเอง
3. แสดง "หลักฐาน" ของคำตอบ: SQL ที่ถูกสร้าง หรือชื่อ Tool/View ที่ถูกเรียก และตารางผลลัพธ์ (raw data)
4. ตอบคำถามได้อย่างน้อย 3 ประเภท
    - คำถามค้นหาข้อมูลทั่วไป เช่น "มีสินค้าอะไรบ้างที่ราคาเกิน 500 บาท"
    - คำถามที่ต้อง JOIN หลายตาราง หรือใช้ View เช่น "ลูกค้าคนไหนสั่งซื้อมากที่สุดเดือนนี้"
    - คำถามเชิงสรุป (Aggregation: COUNT, SUM, AVG, GROUP BY) เช่น "ยอดขายรวมแยกตามหมวดหมู่"

แนวทางการเชื่อมต่อที่อนุญาต (เลือกอย่างน้อย 1 วิธี)

วิธี A: Tool / Function Calling (**แนะนำสำหรับผู้เริ่มต้น)**

- กำหนด Tool ให้ LLM เช่น get_sales_summary(month), search_products(keyword)
- แต่ละ Tool เรียก supabase-js / RPC function ที่เขียนไว้ล่วงหน้า
- ข้อดี: ปลอดภัย ควบคุมได้ ผลลัพธ์แม่นยำ | ข้อจำกัด: ถามได้เฉพาะเรื่องที่มี Tool รองรับ

วิธี B: Text-to-SQL **ผ่าน** Edge Function

- ส่ง Schema ของ View (ชื่อคอลัมน์ + คำอธิบาย) เป็น System Prompt ให้ LLM สร้าง SQL
- Edge Function ตรวจว่าเป็น SELECT เท่านั้น แล้วรันผ่าน RPC function ที่ใช้ Role แบบ read-only
- ข้อดี: ถามได้ยืดหยุ่น | ข้อจำกัด: ต้องตรวจสอบ SQL และจัดการกรณี SQL ผิด

วิธี C: MCP (Model Context Protocol)

- ใช้ Supabase MCP Server (โหมด read-only และจำกัดเฉพาะ project ของกลุ่ม)

หรือเขียน MCP Server เองที่มี Tool สำหรับ query View ของกลุ่ม

- MCP Client/Agent ทำงานฝั่ง Server (เช่น Node.js/Python) แล้วส่งผลให้หน้า Chat บนเว็บ
- หมายเหตุ: Supabase MCP Server ออกแบบสำหรับงานพัฒนา ห้ามเปิด Access Token สู่ผู้ใช้ทั่วไป

วิธีอื่นๆ: เช่น LangChain SQL Agent, n8n, RAG **ด้วย** pgvector — **ทำได้ แต่ต้องผ่านข้อกำหนดความปลอดภัยด้านบน**

LLM **ที่ใช้ได้:** Groq API, Google Gemini, OpenAI, Claude **หรือ** Local LLM (Ollama) — **ระบุในรายงานว่าใช้รุ่นใด**

งานพิเศษ (Bonus ภายใน 5 คะแนน)

- บันทึกประวัติ Chat ลงตาราง chat_logs ใน Supabase (นับเป็นหนึ่งในตารางได้)

การนำเสนอผลงาน

การนำเสนอแบ่งเป็น 2 ส่วน คือ ส่วนงานกลุ่ม และส่วนบุคคล โดยคะแนนการนำเสนอแยกเป็น 2 ส่วนเช่นกัน คะแนนส่วนกลุ่มสมาชิกทุกคนได้เท่ากัน ส่วนคะแนนส่วนบุคคลประเมินแยกเป็นรายคน

ส่วนงานกลุ่ม (ประมาณ 10-15 นาที)

- ภาพรวมระบบ: ที่มา วัตถุประสงค์ ขอบเขต และแผนภาพการทำงาน
- ฐานข้อมูลที่รวมจากสมาชิกทุกคน: ER-Diagram รวม และตารางที่ใช้เชื่อมความสัมพันธ์ (เช่น ตาราง user)
- สาธิตระบบจริงบน Web Hosting: Registration/Login, เพิ่ม/แก้ไข/ลบ ข้อมูล และการแสดงผลจาก View
- สาธิตระบบ Chat: ถามคำถามสดอย่างน้อย 3 ประเภท และแสดง SQL/Tool ที่ถูกเรียก
- สรุปผล ปัญหาที่พบ และข้อเสนอแนะ

ส่วนบุคคล (ประมาณ 5 นาทีต่อคน)

- สมาชิกแต่ละคนนำเสนอฐานข้อมูลส่วนที่ตนออกแบบ: ระดับภายนอก/แนวคิด, ER-Diagram, Relational Schema, Normalization และ Views ของตนเอง
- อธิบาย SQL Script หรือ Query ที่ตนเขียน อย่างน้อย 1 ตัวอย่าง (เช่น View ที่ใช้ JOIN หรือ Aggregation)
- ระบุหน้าที่ของตนในการพัฒนา Web Application และ/หรือระบบ Chat
- ตอบคำถามจากผู้สอนเป็นรายบุคคล (ผู้สอนอาจขอให้แก้ไข Query หรืออธิบายโค้ดของตนสด)

โครงสร้างรายงาน/เอกสารโครงงาน

1. บทนำ – ที่มาและความสำคัญของโครงงาน
2. วัตถุประสงค์
3. ขอบเขตของระบบ
4. การวิเคราะห์ระบบ (System Analysis)
    - แผนภาพการทำงาน (Use Case, DFD หรือ Activity Diagram)
5. การออกแบบฐานข้อมูล (Database Design)
    - การแปลงความหมายระดับภายนอก/ระดับแนวคิด
    - ER-Diagram
    - Relational Schema
    - Normalization (ถ้ามี)
    - SQL Script สำหรับสร้างตาราง, Constraint และ Views
    - SQL สำหรับการวิเคราะห์ข้อมูล (Join, Aggregate, Grouping, Statistical/Mathematical) พร้อมการแปลความหมาย

6. การพัฒนาระบบ (Implementation)
    - ภาษาหรือเครื่องมือที่ใช้ (เช่น HTML + Supabase)
    - ตัวอย่างหน้าจอโปรแกรม
    - การออกแบบระบบ Chat: แผนภาพสถาปัตยกรรม (Web ⟶ Edge Function/MCP ⟶ LLM ⟶ Supabase),
      วิธีที่เลือก (A/B/C), System Prompt, รายการ Tool/View ที่เปิดให้ LLM ใช้
7. การทดสอบระบบ (Testing)
    - กรณีทดสอบ (Test Cases) ของ Login, CRUD และ View
    - กรณีทดสอบ Chat ไม่น้อยกว่า 5 คำถาม: คำถาม | SQL/Tool ที่ถูกเรียก | ผลที่คาดหวัง | ผลจริง | ถูก/ผิด
8. สรุปผลและข้อเสนอแนะ

สิ่งที่ต้องส่ง

1. ไฟล์ pdf รายงานโดยกำหนดเนื้อหาตาม Template
2. ไฟล์ pdf การนำเสนอ (ส่วนงานกลุ่ม และส่วนบุคคลของสมาชิกแต่ละคน รวมไฟล์เดียว)
3. ไฟล์ SQL Script (สร้างตาราง, Constraint, Views, ข้อมูลจำลอง), ไฟล์ analysis_queries.sql และไฟล์ Zip ของ Dataset
4. ลิงก์ Web Application ที่ Deploy แล้ว และลิงก์ Source Code (เช่น GitHub — ห้ามมี API Key ใน repo)
5. วิดีโอสาธิตการใช้งาน (ไม่เกิน 5 นาที) ครอบคลุม Login, CRUD, View และ Chat